Auto Classifier Benchmarking System: Development Plan and IDE-Agent Prompts

Based on your paper, review deck (architecture diagram on slide 12) and requirements slide.

---

0. What you are building (one paragraph)

A web application where a user uploads a tabular dataset, picks the target column, metrics, a time budget and priorities (accuracy vs speed vs interpretability). The system profiles and preprocesses the data, trains ~8 classifiers (Logistic Regression, SVM, Decision Tree, Random Forest, XGBoost, Gradient Boosting, Naive Bayes, KNN) with cross-validation and hyperparameter tuning, measures 5 predictive metrics plus training and prediction time, ranks the models with user-defined weights, **recommends one model with an explanation**, and produces interactive charts and a downloadable report. Every run is stored in a database for experiment tracking.

Your **novelty** (from your research gap slide) is: (1) an integrated recommendation layer, (2) joint multi-metric plus efficiency evaluation, (3) interactive reporting. Protect these three; everything else is supporting machinery.

---

1. Before you start: requirements checklist

1.1 Hardware

Item	Minimum	Recommended
RAM	8 GB	16 GB
CPU	4 cores	8+ cores (parallel CV and Optuna)
Disk	10 GB free	20 GB (datasets, Docker images, results)
GPU	Not needed	Not needed (all models are classical)

1.2 Software and accounts

- Python 3.10 or 3.11 (3.11 is the safest for all libraries)
- Git and a GitHub/GitLab account (private repo, both team members as collaborators)
- IDE with an AI agent (Cursor, VS Code + Copilot/Claude Code, Windsurf, etc.)
- Docker Desktop (for final packaging and PostgreSQL)
- PostgreSQL 15+ (or use SQLite in development and switch later)
- Node.js 18+ **only if** you choose React
- Kaggle account (for API key; some datasets come from Kaggle)

1.3 Python libraries

```
scikit-learn, pandas, numpy, scipy, xgboost, imbalanced-learn, optuna, plotly, matplotlib, seaborn, fastapi, uvicorn, pydantic, sqlalchemy, alembic, psycopg2-binary, streamlit, pytest, ruff, jinja2, openml, joblib, shap (optional), kaleido (for exporting Plotly
```

`images), weasyprint or pdfkit (for PDF reports)`

1.4 Datasets to download first (aim for 10 to 12)

Cover the variety your paper promises: size, class balance, feature types, domain.

Domain	Suggested dataset	Why
Healthcare	Breast Cancer Wisconsin, Pima Indians Diabetes, Heart Disease (UCI)	small, binary
Healthcare (imbalanced)	Stroke Prediction (Kaggle)	strong imbalance
Finance	German Credit, Bank Marketing (UCI), Adult/Census Income	mixed numeric/categorical
Fraud	Credit Card Fraud (Kaggle, 284k rows, 0.17% fraud)	extreme imbalance, large
Customer analytics	Telco Customer Churn (Kaggle)	categorical heavy
Multiclass	Wine Quality, Covertype (sample), Letter Recognition, Dry Bean	multiclass
Via OpenML	`openml.datasets.get_dataset(...)` on any CC18 suite task	standardized, citable

Tip: use the **OpenML-CC18 suite** for a few of them. It makes your results comparable with AMLB (your reference [9]).

1.5 Decisions to freeze in week 1 (write them in `docs/decisions.md`)

These are my recommendations. Change them only deliberately.

1. **Frontend:** FastAPI backend + **Streamlit** frontend for the MVP. Add React only if time remains. Streamlit gets you a working demo in days instead of weeks and is accepted by your slide ("React or Streamlit").
2. **Database:** SQLite in development, PostgreSQL in Docker for the final demo, using SQLAlchemy so the switch is a config change.
3. **Hyperparameter search:** Optuna (TPE sampler) with a fixed trial budget per model, and RandomizedSearchCV as a selectable baseline.
4. **Evaluation protocol:** stratified 5-fold CV, fixed random seed, identical folds for all models, a held-out test set for final scores.
5. **Metrics for multiclass:** macro precision/recall/F1, ROC-AUC with `ovr` and `macro`. Also report weighted versions.
6. **Timing definition:** training time = wall-clock `fit()` time; prediction time = seconds per 1,000 rows, median of repeated runs. Fix `n_jobs` the same for all models and record it.
7. **Ranking method:** weighted sum of min-max normalised scores, plus TOPSIS and a Pareto front as alternatives.
8. **Scope boundary:** tabular classification only (binary and multiclass). Multilabel and multimodal stay in "future work" because your own limitations slide says they're less mature.

1.6 Two technical traps you must plan for (tell the agent about them)

- **Data leakage:** scaling, encoding, imputation and SMOTE must be fit **inside** each CV training fold (use `sklearn.Pipeline / imblearn.Pipeline`). Applying SMOTE before the split inflates every score and invalidates your benchmark.
- **Fair timing:** SVM and KNN explode on large data. You need per-model timeouts, row sub-sampling rules and failure logging (AMLB reports framework failures; you should record failed or timed-out models rather than crashing).

1.7 Team split (2 people)

- **Person A (ML core):** steps 2 to 7 (profiling, preprocessing, models, engine, evaluation, recommendation)
- **Person B (platform):** steps 8 to 11 (DB, reports, API, UI)
- **Together:** steps 0, 1, 12, 13

---

2. Suggested repository structure

```
auto-classifier-benchmark/
├── AGENTS.md                # master context for the IDE agent (Step 0)
├── pyproject.toml
├── docker-compose.yml
├── configs/
│   ├── default.yaml         # models, metrics, CV folds, budgets
│   └── search_spaces.yaml
├── data/
│   ├── raw/  processed/  registry.yaml
├── src/acbs/
│   ├── data/          loader.py, profiler.py, splitter.py
│   ├── preprocessing/ pipeline.py, imbalance.py
│   ├── models/        registry.py, search_spaces.py
│   ├── benchmark/     engine.py, metrics.py, timing.py, tuner.py
│   ├── recommend/     scoring.py, explain.py
│   ├── storage/       models.py, repository.py
│   ├── reporting/     charts.py, report.py, templates/
│   ├── api/           main.py, schemas.py, routes/
│   └── utils/         logging.py, seeds.py
├── app/streamlit_app.py
├── tests/
├── notebooks/         # experiments for the paper
└── docs/
```

---

3. Timeline (10 weeks; compress or stretch as your deadline needs)

Week	Step	Output
1	0, 1	Repo, environment, datasets, registry
2	2, 3	Profiler and leak-free preprocessing
3	4, 5	Model registry, benchmark engine v1 (CLI works end to end)

4	5 (cont.), 6	Tuning, timing, metrics aggregation
5	7	Recommendation engine and explanations
6	8, 9	Database and visualisations and report
7	10	REST API
8	11	Streamlit UI
9	12	Full experiments, validation, tests
10	13	Docker, docs, final paper results, demo

Milestone rule: after Week 4 you should be able to run `python -m acbs.cli run --dataset adult --target income` and get a ranked table. If that works, the project is safe; the rest is packaging.

---

4. Step-by-step plan with IDE-agent prompts

How to use the prompts: paste Step 0 first so the agent knows the project. Then one step at a time. After each step, run the tests, commit, and only then move on. If the agent produces too much, tell it: *"Only implement the files listed. Do not add extra features."*

STEP 0: Project scaffold and agent context (Day 1 to 2)

Goal: a clean repo, reproducible environment, and a permanent context file for the agent.

Prompt 0A (create `AGENTS.md`, or `.cursorrules` for Cursor):

```
Create an AGENTS.md at the repo root describing this project for any AI coding agent.

PROJECT: "Auto Classifier Benchmarking System" (package name: acbs). A system that takes a
tabular dataset + target column, preprocesses it, trains and tunes multiple classifiers
(Logistic Regression, SVM, Decision Tree, Random Forest, XGBoost, Gradient Boosting,
Naive Bayes, KNN) with stratified k-fold cross-validation, measures Accuracy, Precision,
Recall, F1, ROC-AUC, training time and prediction time, ranks models using user-defined
weights, recommends the best model with a textual explanation, stores every experiment in a
database, and presents interactive Plotly charts and a downloadable HTML/PDF report via a
FastAPI backend and Streamlit frontend.

RULES FOR THE AGENT:
- Python 3.11, type hints everywhere, docstrings on public functions, ruff-formatted.
- Never apply scaling/encoding/imputation/SMOTE outside a sklearn/imblearn Pipeline;
  all of it must be fit only on training folds (no data leakage).
- All randomness goes through a single seed from config (default 42).
- All models must be compared on identical CV folds.
- Never crash the whole benchmark when one model fails or times out; record status
  ("ok" | "failed" | "timeout") and the error message and continue.
- Config lives in configs/*.yaml, never hardcoded.
- Every module gets pytest tests in tests/ mirroring the src structure.
- Use logging, not print.
- Do not add dependencies without adding them to pyproject.toml.
- Keep functions small and pure where possible; business logic must not live in API or UI files.
```

```
Also include: repo structure (use the tree I give below), common commands (install, test,
lint, run API, run UI), and a "definition of done" checklist for any change.
[paste the repository tree from section 2 of the plan]
```

Prompt 0B (scaffold):

```
Read AGENTS.md. Scaffold the repository exactly as described: create all folders with
__init__.py files, a pyproject.toml (use hatch or setuptools, package "acbs" under src/),
dependencies: scikit-learn, pandas, numpy, scipy, xgboost, imbalanced-learn, optuna, plotly,
matplotlib, seaborn, fastapi, uvicorn[standard], pydantic, pydantic-settings, sqlalchemy,
alembic, streamlit, pyyaml, jinja2, openml, joblib, typer; dev deps: pytest, pytest-cov, ruff,
mypy, pre-commit. Add: .gitignore (data/raw, __pycache__, .venv, *.db), a Makefile with
targets install/test/lint/api/ui, a ruff + pre-commit config, a GitHub Actions workflow that
runs ruff and pytest, and a minimal CLI entry point `acbs` (typer) with a `version` command.
Add one smoke test. Do not implement any business logic yet. Show me the commands to
create the venv and run the tests.
```

Done when: `make test` passes, CI is green, both teammates can clone and run.

STEP 1: Dataset registry and loader (Week 1)

Goal: one consistent way to load any benchmark dataset.

Prompt 1:

```
Implement src/acbs/data/loader.py and data/registry.yaml.

1. registry.yaml: list datasets with fields: name, source ("openml"|"uci"|"kaggle"|"local"),
   identifier (OpenML id / URL / path), target_column, domain (healthcare|finance|fraud|
   customer|general), task ("binary"|"multiclass"), and notes.
   Add entries for: breast_cancer, pima_diabetes, heart_disease, german_credit,
   bank_marketing, adult, telco_churn, credit_card_fraud, wine_quality, dry_bean,
   covertype_sample, stroke. Use OpenML ids where they exist.
2. loader.py:
   - load_dataset(name_or_path, target=None) -> LoadedDataset (dataclass: X: DataFrame,
     y: Series, meta: dict) with caching to data/processed/ as parquet.
   - Supports OpenML (via openml package), local CSV/Excel/Parquet uploads, and registry names.
   - Validates: target exists, at least 2 classes, no all-null columns, minimum 50 rows;
     raises clear custom exceptions (DatasetError subclasses).
   - Strips whitespace from column names, parses obvious numeric-as-string columns,
     encodes the target with LabelEncoder and keeps the mapping in meta.
3. A script scripts/download_datasets.py that downloads everything in the registry.
4. Tests using small in-memory CSVs: valid load, missing target, single-class target,
   duplicate column names.
Do not do any preprocessing here (that is a later step).
```

Done when: `download_datasets.py` pulls all datasets and `load_dataset("adult")` returns X, y.

STEP 2: Dataset profiler and meta-features (Week 2)

Goal: the "analyzes data characteristics" part of your abstract. It also feeds explanations later.

Prompt 2:

```
Implement src/acbs/data/profiler.py.

profile_dataset(X, y) -> DatasetProfile (pydantic model, JSON serializable) containing:
- n_rows, n_features, n_numeric, n_categorical, n_high_cardinality_categorical (>50 unique)
- missing_ratio overall and per column (top 10 worst), n_duplicate_rows
- class_distribution (counts and ratios), n_classes, imbalance_ratio (majority/minority),
  is_imbalanced (ratio > 3 configurable), task_type (binary|multiclass)
- numeric stats: skewness summary, share of features with |skew|>1, outlier share (IQR rule)
- dimensionality ratio (features/rows), estimated memory MB
- size_category: small (<5k rows), medium (5k-100k), large (>100k)
- warnings: list of human-readable strings (e.g. "Severe class imbalance 1:577",
  "3 columns have >40% missing values", "Possible ID column: <name>")
Also detect likely ID columns (unique ratio > 0.95 and integer/string) and constant columns,
and expose them as `suggested_drop_columns`.
Add a function profile_to_dict() and tests on synthetic data (balanced, imbalanced,
with missing values, with an ID column).
```

STEP 3: Leak-free preprocessing pipeline (Week 2)

Goal: data cleaning, feature engineering and imbalance handling per your "Data Preprocessing Module".

Prompt 3:

```
Implement src/acbs/preprocessing/pipeline.py and imbalance.py.

build_preprocessor(profile, config, model_family) -> sklearn ColumnTransformer:
- numeric: SimpleImputer(median) -> optional outlier clipping -> scaler. Scaling is applied
  only for model families that need it (linear, svm, knn, naive_bayes-gaussian optional);
  tree-based models (decision tree, RF, GB, XGBoost) skip scaling.
- categorical: SimpleImputer(most_frequent) -> OneHotEncoder(handle_unknown="ignore",
  min_frequency configurable) for cardinality <= 15; for higher cardinality use
  TargetEncoder (sklearn >= 1.3) or OrdinalEncoder. Make this threshold configurable.
- drop ID/constant columns from profile.suggested_drop_columns.
- optional feature selection (SelectKBest or variance threshold) controlled by config.
imbalance.py: get_imbalance_strategy(profile, config) returns one of "none" |
"class_weight" | "smote" | "random_under". Default rule: class_weight when imbalance
ratio > 3; SMOTE only if user enables it and minority count >= 6; never SMOTE on
datasets above a configurable row limit.
build_full_pipeline(model, preprocessor, imbalance_strategy) returns an
imblearn.pipeline.Pipeline where SMOTE (if used) is placed AFTER the preprocessor and only
runs on training folds. If strategy is class_weight, set class_weight="balanced" on
models that support it (and sample_weight handling note for XGBoost via scale_pos_weight).
Also create splitter.py with: make_holdout_split (stratified, 80/20) and
make_cv_splitter (StratifiedKFold, shuffled, seeded) so every model gets identical folds.
Write tests proving: no leakage (fit on train only, via checking transformer statistics),
unseen categories at predict time don't crash, tree models get no scaler, SMOTE runs only
inside fit.
```

STEP 4: Model registry and search spaces (Week 3)

Goal: your 8 algorithms as plug-in components.

Prompt 4:

Prompt 4:

```
Implement src/acbs/models/registry.py, search_spaces.py and configs/search_spaces.yaml.

registry.py:
- A ModelSpec dataclass: name, family ("linear"|"svm"|"tree"|"ensemble"|"boosting"|
  "naive_bayes"|"instance"), estimator_factory(params, seed, n_jobs), needs_scaling: bool,
  supports_class_weight: bool, supports_proba: bool, interpretability_score (1-5),
  scalability_note, max_rows (optional safety limit, e.g. SVM 20000, KNN 100000).
- register 8 models: LogisticRegression, SVC (probability=True; if rows > max_rows use
  LinearSVC wrapped in CalibratedClassifierCV), DecisionTree, RandomForest, XGBoost,
  GradientBoosting (sklearn), GaussianNB, KNN.
- get_models(names=None) and an easy decorator/function to register extra models later.
search_spaces.py + YAML: Optuna search space per model with sensible ranges, e.g.
LR: C log-uniform 1e-3..1e2, penalty/solver combos; SVM: C, gamma, kernel (rbf/linear);
DT: max_depth, min_samples_split, min_samples_leaf, criterion; RF: n_estimators 100-500,
max_depth, max_features, min_samples_leaf; XGB: n_estimators, learning_rate log, max_depth,
subsample, colsample_bytree, reg_lambda; GB: n_estimators, learning_rate, max_depth,
subsample; NB: var_smoothing log; KNN: n_neighbors, weights, p.
Provide suggest_params(trial, model_name) for Optuna AND a RandomizedSearchCV-compatible
param_distributions dict from the same definition (single source of truth).
Tests: every registered model can fit and predict_proba on a toy dataset, and every search
space can be sampled.
```

STEP 5: Benchmark engine: training, tuning, metrics, timing (Week 3 to 4, the heart of the project)

Goal: the "Model Training & Benchmarking Module" and "Evaluation Module".

Prompt 5A (metrics and timing):

```
Implement src/acbs/benchmark/metrics.py and timing.py.

metrics.py: compute_metrics(y_true, y_pred, y_proba, classes) -> dict with accuracy,
precision_macro, recall_macro, f1_macro, precision_weighted, recall_weighted, f1_weighted,
roc_auc (binary: roc_auc_score on positive-class proba; multiclass: multi_class="ovr",
average="macro"), plus balanced_accuracy and MCC. Handle edge cases: a class missing from a
fold (roc_auc -> NaN, not a crash), zero_division=0, models without predict_proba
(fall back to decision_function with softmax, flag it). Also provide get_scorers() returning
the sklearn scorer dict for cross_validate.
timing.py: measure_training_time (context manager using time.perf_counter) and
measure_prediction_time(model, X, repeats=5, per=1000) returning median seconds per 1000
rows after one warm-up call. Record n_jobs used. Add unit tests with a dummy estimator
using time.sleep to verify timings within tolerance.
```

Prompt 5B (engine):

```
Implement src/acbs/benchmark/engine.py and tuner.py.

BenchmarkConfig (pydantic): models list, cv_folds=5, seed, metrics_primary="f1_macro",
tuning ("none"|"optuna"|"random"), n_trials per model (default 20), time_budget_seconds
per model (default 300) and total budget, n_jobs, imbalance_strategy override, test_size.

tuner.py: tune_model(model_spec, X_train, y_train, preprocessor_factory, cfg) -> best_params,
trials_history. Use Optuna TPE (seeded), optimize the primary metric via cross_val_score on
the SAME CV splitter, enforce a timeout, and use a MedianPruner where applicable. Provide the
```

```
RandomizedSearchCV alternative behind the same interface.

engine.py: run_benchmark(dataset: LoadedDataset, cfg, progress_callback=None) ->
BenchmarkResult:
 1. profile the dataset; hold-out split.
 2. for each model: build the full pipeline, tune (if enabled) on train only, then run
    cross_validate with return_train_score=False on the training set to get per-fold metrics
    and fit_time, then refit on full train, evaluate on the hold-out test set, and measure
    final training time and prediction time (per 1000 rows).
 3. Wrap each model in try/except + timeout (use a subprocess/joblib timeout or Optuna
    timeout + a guard); on failure store status and error and continue.
 4. Skip or subsample models that exceed ModelSpec.max_rows, and record that decision in a
    "notes" field (never silently).
 5. Call progress_callback(model_name, step, total, message) so the UI can show progress.
 6. Return per-model: cv mean/std for every metric, test metrics, training_time,
    prediction_time_per_1k, best_params, status, notes, plus dataset profile and config
    (for reproducibility). Include library versions and seed in the result.
Parallelism: run folds in parallel with n_jobs but keep models sequential so that timings are
comparable. Document this decision in a docstring.
Add a CLI command `acbs run --dataset adult --target income --models all --tuning optuna`
that prints a results table (use rich or tabulate).
Tests: run the engine on a small synthetic dataset with 3 fast models and tuning="none";
check result structure, determinism with the same seed, and that a deliberately broken
model gets status="failed" without stopping the run.
```

Done when: the CLI command above produces a table on the Adult dataset in a reasonable time and results are identical across two runs with the same seed.

STEP 6: Results aggregation and statistical comparison (Week 4)

Goal: make the comparison scientifically defensible (this is what your reviewers will probe).

Prompt 6:

```
Implement src/acbs/benchmark/aggregate.py.

- results_to_dataframe(BenchmarkResult) -> tidy DataFrame, one row per model.
- Per-fold score matrix extraction for statistical tests.
- compare_models_statistically(fold_scores): Friedman test across models when >= 3 models
  and >= 5 folds is not enough power, so also provide pairwise corrected resampled t-test
  (Nadeau-Bengio) and a Wilcoxon signed-rank fallback; return p-values and a flag
  "statistically_tied_with_best".
- For multi-dataset experiments (used later in step 12): compute average rank per model
  across datasets and a Nemenyi/critical-difference style summary.
- rank_by_metric(df, metric) with ties handled by average rank.
Tests with synthetic fold scores where one model is clearly better and where two are tied.
```

STEP 7: Recommendation engine (Week 5, your main novelty)

Goal: turn numbers into "use model X, because...".

Prompt 7:

```
Implement src/acbs/recommend/scoring.py and explain.py.
```

```
UserPreferences (pydantic): weights for accuracy/f1/precision/recall/roc_auc/train_time/
predict_time/interpretability (non-negative, auto-normalised to sum 1), constraints
(max_training_time_s, max_prediction_time_ms_per_1k, min_recall, min_precision), method
("weighted_sum"|"topsis"), preset ("balanced"|"max_accuracy"|"fast_inference"|
"interpretable"|"imbalance_safe" where imbalance_safe emphasises recall and F1).

scoring.py:
- normalize(): min-max for benefit metrics, inverted min-max for cost metrics (time),
  use log-scale for time before normalising so one very slow model doesn't flatten the rest.
- score_models(results_df, prefs) -> DataFrame with a composite score and rank; apply hard
  constraints first (violating models are marked "excluded" with the reason).
- TOPSIS implementation as an alternative.
- pareto_front(results_df, x="training_time", y="f1_macro") returns the non-dominated models.
- sensitivity_analysis(results_df, prefs): perturb weights +-20% repeatedly and report how
  often each model stays rank 1 (a "recommendation stability" percentage).
explain.py: generate_explanation(rec, results_df, profile, prefs) -> structured object with
a headline ("Recommended: XGBoost"), a 3-5 sentence plain-English reason built from templates
(strongest metrics vs runner-up, time trade-offs, whether it is statistically tied with the
runner-up, stability %), caveats (e.g. "Dataset is highly imbalanced; accuracy is
misleading, ranking emphasised F1"), and a "why not" line for 2 other models. Use
deterministic templates, NOT an LLM, so results are reproducible.
Tests: changing weights from accuracy to speed changes the winner on a crafted results table;
constraints exclude models; the Pareto front is correct on a hand-made example.
```

STEP 8: Experiment tracking database (Week 6)

Goal: persistent storage (PostgreSQL/MySQL on your slide).

Prompt 8:

```
Implement src/acbs/storage/models.py and repository.py with SQLAlchemy 2.0 and Alembic.

Tables: datasets (id, name, source, n_rows, n_features, profile_json, file_hash, created_at),
experiments (id, dataset_id, config_json, preferences_json, status, started_at, finished_at,
seed, lib_versions_json), model_runs (id, experiment_id, model_name, status, error, notes,
cv_metrics_json, test_metrics_json, training_time, prediction_time_per_1k, best_params_json,
fold_scores_json), recommendations (id, experiment_id, model_name, score_table_json,
explanation_json, stability_pct).
Use SQLite by default and a DATABASE_URL env var for PostgreSQL. Provide a Repository class
with: create_experiment, update_status, save_results, save_recommendation, get_experiment,
list_experiments(filters), delete_experiment, and load_result_as_dataframe. Create the first
Alembic migration. Dedupe uploaded datasets by file hash. Write tests against an in-memory
SQLite database. No business logic in this layer.
```

STEP 9: Visualisation and report generation (Week 6)

Goal: "Visualization & Reporting Module".

Prompt 9:

```
Implement src/acbs/reporting/charts.py, report.py and templates/report.html.j2.

charts.py (all return plotly Figure objects):
```

```
- performance_bar(df, metric) and grouped_metrics_bar(df)
- metric_radar(df, models, metrics) (normalised 0-1)
- time_comparison(df) (training vs prediction time, log y-axis option)
- accuracy_vs_time_scatter(df) with the Pareto front highlighted and bubble size = F1
- cv_boxplot(fold_scores) showing variance across folds
- confusion_matrix_heatmap(y_true, y_pred) for a chosen model (store hold-out predictions
  in the results so this works)
- roc_curves(...) for binary tasks, and a feature-importance chart for tree/linear models
- score_breakdown_stacked_bar(scored_df) showing each metric's weighted contribution to the
  final score (this explains the recommendation visually)
report.py: build_html_report(experiment) -> a single self-contained HTML file (embed Plotly
via include_plotlyjs="cdn" or inline option) with sections: dataset summary and warnings,
configuration, results table, charts, recommendation and explanation, failed/skipped models,
reproducibility info (seed, versions). Add export_pdf(experiment) using weasyprint (optional
dependency; degrade gracefully if missing) and export_csv/json.
Use a clean, consistent colour palette and accessible colour choices. Tests: every chart
function returns a Figure on a fixture results DataFrame; the report renders without errors
and contains the recommended model name.
```

STEP 10: REST API (Week 7)

Goal: backend for the web app.

Prompt 10:

```
Implement the FastAPI app in src/acbs/api/ (main.py, schemas.py, routes/).

Endpoints:
- POST /datasets (multipart upload; returns dataset id + profile + warnings; max size from
  config) and GET /datasets, GET /datasets/{id}, GET /datasets/registry (built-in ones)
- POST /experiments (dataset id, target, BenchmarkConfig, UserPreferences) -> returns
  experiment id immediately and runs the benchmark in a background worker (use
  concurrent.futures ProcessPoolExecutor or Celery-less background tasks; keep it simple but
  not blocking the event loop)
- GET /experiments/{id}/status (progress %, current model, messages) -- store progress in the DB
- GET /experiments/{id}/results, GET /experiments/{id}/recommendation
- POST /experiments/{id}/rerank (new preferences, NO retraining; recompute the ranking only)
- GET /experiments/{id}/report?format=html|pdf|csv|json
- GET /experiments (history), DELETE /experiments/{id}
- GET /health
Use Pydantic request/response models, proper HTTP error codes, CORS, request-size limits,
filename sanitisation on upload, and structured logging. Endpoints must be thin and call the
service layers from earlier steps. Add API tests with TestClient covering: upload -> run on a
small dataset -> poll -> get results -> rerank with different weights returns a different
ranking.
```

Note: `/rerank` is important. Users can change priorities and instantly see a new recommendation without retraining. This is a strong demo moment.

STEP 11: Streamlit user interface (Week 8)

Goal: the "User Interface (Web Application)" box in your diagram.

Prompt 11:

```
Build the Streamlit frontend in app/streamlit_app.py plus app/pages/ and app/components/.
It talks ONLY to the FastAPI backend through a small api_client.py (requests).

Pages:
1. "New Benchmark": choose a built-in dataset or upload CSV; show a dataset profile card
   (rows, features, types, missing %, class balance chart, warnings); select the target
   column (auto-suggest last column), select models (multiselect, all by default), metrics,
   CV folds, tuning method and trial count, time budget; priorities via a preset dropdown AND
   advanced sliders for weights; optional constraints. A "Run benchmark" button.
2. "Progress": poll the status endpoint, show a progress bar and per-model status table
   (ok/failed/timeout/running) with st.status or auto-refresh.
3. "Results": leaderboard table (sortable, with conditional colour), tabs for Performance,
   Time analysis, Radar, Accuracy-vs-Time Pareto, CV variance, and a model-detail view
   (confusion matrix, ROC, feature importance, best params). The recommendation card at the
   top shows the winning model, the explanation text, and the stability percentage. Changing
   the weight sliders calls /rerank and updates the ranking and charts live.
4. "History": list past experiments with filters, open, delete, and compare two experiments.
5. "Export": download HTML, PDF, CSV, JSON.
Include loading states, friendly error messages, and a sidebar with a short help text for
each metric. Keep UI code free of ML logic. Add a docs/ui_walkthrough.md with screenshots
placeholders. Provide a manual test checklist for the page flow.
```

(If you choose React later, keep the FastAPI contract unchanged, so you only replace this step.)

STEP 12: Experiments, validation and testing (Week 9, produces your paper's results)

Goal: prove the system works and generate evidence for the paper. Do not skip this; this is what turns a "tool" into a "research project".

Prompt 12A (full experiment runner):

```
Create scripts/run_full_experiments.py and notebooks/results_analysis.ipynb.

The script runs the benchmark on all datasets in data/registry.yaml with three user
preference presets (balanced, max_accuracy, fast_inference), fixed seed 42, 5-fold CV and
Optuna with 20 trials, saves everything to the DB and to results/*.csv. It must be resumable
(skip finished dataset+config pairs), log failures, and print a final summary.
The notebook loads the results and produces paper-ready figures and tables: (1) mean
rank of each model across datasets with a critical-difference style plot, (2) accuracy vs
training-time scatter per dataset, (3) a table showing how the recommended model CHANGES
with preferences (this directly supports your novelty claim), (4) performance on imbalanced
datasets showing accuracy vs F1/recall divergence, (5) recommendation stability across
datasets, (6) runtime scaling vs dataset size.
Export figures as 300-dpi PNG/SVG into results/figures/.
```

Prompt 12B (validation against a baseline, optional but strong):

```
Add scripts/compare_with_automl.py that, for 4-5 datasets, runs FLAML (or auto-sklearn
if installable, otherwise AutoGluon-Tabular) with the same time budget and the same hold-out
split as our system, and compares the hold-out F1/accuracy of (a) our top-1 recommended model
and (b) the AutoML framework's best model. Output a CSV and a figure. The goal is to show
that our recommended model is competitive (not necessarily better) while also providing
multi-metric trade-offs and explanations that the AutoML tool does not. Document the
```

```
multi-metric trade-offs and explanations that the AutoML tool does not. Document the
fairness setup (same split, same budget, same seed) in docs/validation.md.
```

Prompt 12C (test hardening):

```
Review the whole repo for gaps. Increase test coverage to >= 80% on src/acbs. Add: property
tests for scoring normalisation, regression tests with stored golden results for a tiny
dataset, a data-leakage test, tests for edge cases (single-feature dataset, 2-class highly
imbalanced dataset with 10 minority samples, all-categorical dataset, dataset with a
constant column, string target labels, numeric target with 50 unique values -> reject as
regression), and a performance smoke test. List any bugs you find and fix them.
```

STEP 13: Packaging, documentation and final deliverables (Week 10)

Prompt 13A (Docker):

```
Create a Dockerfile (multi-stage, python:3.11-slim) and docker-compose.yml with services:
api (FastAPI), ui (Streamlit), db (PostgreSQL 15 with a volume), all configured via a
.env.example. Run Alembic migrations on API startup. Add healthchecks and a make target
`make up`. Verify that `docker compose up` yields a working app at localhost with a
built-in dataset benchmark running end to end. Document it in README.
```

Prompt 13B (docs):

```
Write README.md (overview, architecture diagram in mermaid matching our modules, quick
start with and without Docker, CLI examples, configuration reference, how to add a new model,
how the recommendation scoring works, limitations), docs/architecture.md, docs/api.md
(generated from the OpenAPI schema), and docs/reproducibility.md. Add a CITATION-style note.
Include a "Known limitations" section: tabular classification only, hold-out size,
timing depends on hardware, no multilabel/multimodal support.
```

Final deliverables checklist

- ☐ Working demo (Docker or local), with 2 prepared datasets for the live demo
- ☐ Source repo with README and tests passing in CI
- ☐ Results tables and figures from Step 12
- ☐ Updated paper: add Methodology, Implementation, Results, Discussion, Conclusion, Future Work sections to the draft that currently has only Abstract, Introduction and References
- ☐ Demo video (3 to 5 minutes) as a backup in case of live-demo failure
- ☐ Final presentation slides (fill the empty "Project work plan and timeline" slide using Section 3 above)

---

5. Risks and how to handle them

Risk	Mitigation
Benchmark takes too long (SVM/KNN on large data)	per-model timeout, `max_rows` rule, sub-sampling note, 20 Optuna trials default, "quick mode" for demos

Inflated scores from leakage	pipelines only; leakage test in Step 3
Agent writes huge, unfocused code	one step per prompt, "only implement listed files", commit after every step
Results not reproducible	single seed, stored config and lib versions, identical folds
Streamlit feels slow when rerunning	cache API results with `st.cache_data`; use `/rerank` instead of retraining
Scope creep (deep learning, multilabel, NLP)	keep in "Future Work"; your own slide lists them as limitations
Timings noisy	median of repeats, warm-up call, fixed `n_jobs`, close other apps while running experiments

---

6. Optional extensions (only after Step 12 is complete)

1. Meta-learning recommender: train a small model on (dataset meta-features, best model) pairs from your experiments to predict a good model **before** training everything (ties to your references [5] and [14]).
2. SHAP-based explanations for the recommended model.
3. Regression support (extend the same architecture).
4. Authentication and multi-user workspaces.
5. React frontend replacing Streamlit.
6. Threshold tuning for imbalanced datasets (cost-sensitive decision threshold).

---

7. Working method with an IDE agent (practical tips)

1. Always start a session by telling the agent: "*Read AGENTS.md and the files in src/acbs/ first.*"
2. One prompt = one module. Review the diff before accepting. Run tests yourself.
3. If something fails, paste the **full error and the failing test** back, not a summary.
4. After each step, ask: "*Review the code you just wrote for data leakage, unhandled exceptions, hardcoded values, and missing tests. List issues, then fix them.*"
5. Keep a `docs/decisions.md` log. It becomes raw material for the Methodology section of your paper.
6. Commit with clear messages per step (`feat(benchmark): add metrics and timing`).